Tóth Avar Áron, DQMKMR

Sakk fejlesztői dokumentáció

1. A Program Fő Célja és Működési Elve

A "Sakk TUI++" egy interaktív, C++ nyelven megírt, objektumorientált sakkprogram, amely egyedülálló terminálos grafikus felületet (TUI) kínál az `econio` keretrendszer felhasználásával. A program célja nem csupán egy egyszerű sakkjáték megvalósítása volt, hanem egy elemző játékmóddal rendelkező rendszer létrehozása, ahol a felhasználó a mérkőzés bármely pontján visszaléphet a történetben, új ágat nyithat (alternatív elágazásokat teremtve a játszmának), és ezeket az állapotokat fájlba is elmentheti.

A program kódja dinamikus memóriakezeléssel, szivárgásmentesen (Memtrace által ellenőrizve) operál.

2. Objektumorientált Tervezés (OOP Elvek)

2.1 Heterogén Kollekció és Polimorfizmus

A sakktábla (`Piece* board[8][8]`) egy heterogén kollekció. Maga a tábla kizárólag a virtuális ősosztály (`Piece`) pointereit tárolja. A polimorfizmus révén a fő logika anélkül tudja mozgatni, másolni és ellenőrizni a bábukat, hogy ismerné azok pontos típusát.

- **Polimorf viselkedés:** Az ősosztály `is_move_valid()` metódusa tisztán virtuális (absztrakt). Ezt minden konkrét bábu (`King` , `Queen` , `Rook` , stb.) saját maga implementálja a sakk lépésszabályainak megfelelően.

- **Virtuális Destruktor:** A `Piece` osztály kötelezően virtuális destruktorral rendelkezik, biztosítva, hogy a heterogén kollekcióból való törléskor (`delete p;`) a leszármazott osztályok esetleges destruktora is helyesen lefusson, megelőzve a memóriaszivárgást.

2.2 Clone() tervezési minta (Prototype)

Mivel a tábla állapotait folyamatosan menteni kell (a fába lépésenként), a heterogén kollekció másolása kritikus probléma volt. Ezt a `clone()` virtuális metódussal oldottam meg. Minden bábu képes saját magáról egy tökéletes dinamikus másolatot készíteni anélkül, hogy a hívó fél tudná, milyen báburól van szó (pl. `board[r][c]->clone()`).

Tóth Avar Áron, DQMKMR

3. Szoftverarchitektúra: Az MVC Minta Megvalósulása

A kód olvashatóságának és bővíthetőségének érdekében a program logikája az MVC (Model-View-Controller) elvei alapján van szétválasztva:

- **Model (Sakk logika):** A `chess.cpp` tartalmazza a bábuk mozgásszabályait (is_move_valid), a sakk/patt/sáncolás és en-passant ellenőrzést. A `storage.cpp` pedig magát a fő adatszerkezetet (a lépéstörténeti fát) valósítja meg.
- **View (Megjelenítés):** A `display.h` és `display.cpp` fájlok felelnek a grafikus felületért. Ezek az `econio` nyers terminálkezelését használják a tábla színes kirajzolásához, a menük és bemeneti mezők megjelenítéséhez. A View nem tartalmaz sakk-logikát.
- **Controller (Vezérlés):** A `main.cpp` köti össze a Modelt és a View-t. Ez tartalmazza az Állapotgépet (State Machine), amely a menüpontok, a játékmódok és az elemző mód közötti navigációt irányítja, beolvassa a View-tól kapott interakciókat, és frissíti a Model állapotát.

4. A Fő Adatstruktúra: Dinamikus Fa (N-ary tree)

A program legkomplexebb része a memóriában felépülő lépéstörténet. Mivel az elemző módban egy korábbi pozícióból újabb lépéseket tehetünk (amivel a történelem elágazik), egy egyszerű láncolt lista nem volt elegendő.

Ezt a problémát egy **dinamikus n-áris fával** (`BoardTree`) oldottam meg. A fa minden csomópontja (`Board` struktúra) tartalmaz:

- Egy 8x8-as heterogén kollekciót (a konkrét táblaállapotot).

- Egy pointert a szülő csomópontra (visszalépéshez).
- Egy dinamikusan nyújtott pointer tömböt a gyermek csomópontokra (`next` array), amelyek az alternatív jövőbeli lépéseket reprezentálják.

A `BoardTree` osztály teljesen önálló RAII memóriakezelést végez (saját destruktora van, mely rekurzívan felszabadítja az egész fát).

5. Fájlkezelés

A játékállapotok elmentése és visszatöltése a `storage.cpp`-ben történik. Mivel az adatszerkezet egy bonyolult mutatókból álló fa, a mentés egy speciális, egyedi formátumban történik (pre-order bejárással). A fájlba minden csomópontnál kimentésre kerül a tábla pontos állapota (a bábuk típusa, színe), valamint a fában lévő gyermekek száma.

A visszatöltéskor a `load` metódus ugyanilyen logika mentén rekurzívan újraépíti a dinamikus struktúrát a memóriában, tökéletesen reprodukálva a mentett elágazásokat.

> **6. Kivételkezelés (Exception Handling)**
>
> A C++ kivételkezelési mechanizmusát elsősorban a fájlkezelési műveleteknél alkalmaztam. Ha egy mentés betöltése során a fájl nem létezik, vagy a formátuma érvénytelen, a rendszer egy saját, C++ standard-kompatibilis kivételt (`FileNotFoundException`, amely az `std::exception`-ből származik) dob. Ezt a kontroller rész elkapja (`try-catch`), és egy elegáns hibaüzenetet rajzol ki a grafikus felületen, ahelyett, hogy a program összeomlana.

Tóth Avar Áron, DQMKMR

7. Összegzés és Megbízhatóság

A fejlesztés során kiemelt figyelmet kapott a memóriabiztonság (a Memtrace eszközzel folyamatosan tesztelve) és a szélsőséges (edge) esetek lekezelése. Minden bonyolult sakk-mechanika (En Passant menet közbeni ütés, király sáncolása, patthelyzet felismerése, és gyalogáttörés/promóció) implementálva van és masszív egységtesztek (Unit Tests) fedik le a `teszt.cpp`-ben. A kódbázis tiszta, jól strukturált, és minden szoftverfejlesztési elvárásnak megfelel.